GALILEO Graph Databases Group Second Deadline Report (19 November 2025)

Francesco Pivotto 2158296
Giorgia Amato 2159999
Alessio Demo 2142885

November 19, 2025

Contents

1 Introduction

1.1 Context and Objective

The preceding project phase involved the **assessment of query execution** on the database for each dataset, followed by the processing of the results obtained from our implementation using the `galois_eval.py` script.

The primary objective of the current task is to detail the **development process** implemented to replicate the following **baselines**, specifically utilizing the **Internal Knowledge datasets**:

- **SQL Baseline**: The system directly prompts the *Large Language Model* (LLM) with an **SQL query** as the input.
- **NL (Natural Language) Baseline**: The system employs a **Natural Language prompt** as the direct input to the LLM.

1.2 Report Structure

This introductory section establishes the methodological context of the project:

- Section 2 details the project's core: the Unified Architecture and Execution Flow common to both baselines. This section describes the software design patterns adopted for modularity, the standardized execution pipeline and the definitions for the data context and output format (FULL JSON).
- Section 3 describes the specific implementations for the two evaluation modes, the NL Baseline and the SQL Baseline. Leveraging the common architecture, this section focuses on how each baseline handles its unique input (a natural language prompt or an SQL query) within the unified execution flow.
- Section 4 show, for each implemented baseline, **synoptic tables** reporting the derived evaluation measures and metrics. It analyzes and compares key metrics (F1-CELL, CARDINALITY, TUPLE-CONSTRAINT) and costs (tokens and latency) for both baselines across the different LLM providers tested.
- Section 5 outlines Future Work that will be made in the next tasks.
- Section 6 details the individual contributions of each team member to the task's development.

2 Core Architectural Design and Execution Flow

2.1 Architectural Design Patterns for Modularity

The codebase's central logic is structured around established software design patterns to ensure high modularity, extensibility, and maintainability across heterogeneous components.

Adapter Pattern (`llm_wrappers.py`): This pattern standardizes the interface for interacting with heterogeneous LLM providers (e.g., Gemini, Watsonx). By abstracting the provider-specific connection details, it enables the core execution engine to interact with all models through a uniform API.

Factory Pattern (`llm_factory.py`): Implemented as a simple factory method, this pattern decouples client components (e.g., `sql_baseline.py`, `nl_baseline.py`) from the specific instantiation logic of concrete LLM wrapper objects. Extensibility to new providers is achieved solely by updating the internal `PROVIDER_MAP` registry.

Template Method Pattern (`LLMBaseWrapper` in `llm_wrappers.py`): The abstract `LLMBaseWrapper` defines the invariant sequence (the *template*) for the life cycle of the LLM instance retrieval. It establishes the mandatory sequence while deferring the implementation of the provider-specific connection details (via `create_llm_instance()`) to concrete subclasses.

Chain/Pipeline Pattern (LCEL) (`build_lcel_chain.py`): This pattern, realized using LangChain Expression Language (LCEL), defines a structured, sequential execution flow:

Context Generation → Prompt Construction → LLM Invocation

This structured approach ensures predictable and auditable data transformation across the execution path.

2.2 Unified Architecture and Execution Workflow (The Pipeline)

To ensure high modularity, extensibility, and maintainability, we adopted a unified base architecture that serves both baselines (NL and SQL). This architecture centralizes the execution logic, abstracting interactions with LLM providers and standardizing the data flow.

The execution workflow is structured into a four-stage pipeline, ensuring robust and consistent processing across all baseline types:

1. **Context Information Retrieval**: The `get_db_context` function dynamically retrieves the necessary database context that will be attached to the LLM prompt together with the corresponding SQL query fro SQL baseline or NL prompt for NL baseline. For instance this includes retrieving the database schema for SQL/NL baselines.

2. **Prompt Construction and Injection**: The derived context is programmatically injected into a sophisticated `ChatPromptTemplate`. This stage manages the insertion of system instructions and the baseline-specific input (the NL or SQL query), and also the enforce the required JSON output structure via an `PydanticOutputParser` object.

3. **LLM Invocation**: The fully prepared and structured prompt is transmitted to the configured LLM instance, which was instantiated via the Factory/Adapter layers.

4. **Response Standardization**: The `parse_llm_response` function enforces cross-provider output standardization: This involves the mandatory validation and parsing of the requisite JSON structure returned by the LLM. It then enriches this output into the `FULL_JSON` format by extracting the `token consumption` from the LLM's response metadata and internally calculating and appending the `LLM response latency`.

This last standardized and enriched data process is essential for downstream performance evaluation, furthermore thanks to this centralized architecture, the execution logic is provider-independent, and the workflow is uniform.

2.3 Data Context and Output Standardization

This section defines the two primary contexts used to condition the Large Language Models (LLMs) and establishes the **standardized JSON output format** mandatory for all baseline evaluations (NL, SQL, and Palimpzest).

2.3.1 Data Context Categorization

The evaluation utilizes two distinct dataset contexts, governing the type of factual information provided to the LLM during prompting:

- **Internal Knowledge (IK):** In these datasets, the model relies exclusively on its pretrained knowledge and linguistic reasoning capabilities. The prompt includes only the natural language question and the database schema for both the **SQL** and **NL** baselines, but no factual data retrieved from the database.

- **Model Context (MC):** These datasets are designed to enrich the prompt with external information retrieved from a knowledge base. For this specific project task, MC datasets were **not used** in either the NL or SQL baselines as indicated in GALOIS paper, but could represents a good resource for RAG purposes.

2.3.2 Standardized Output Format

Every response produced by any baseline is written in a JSON file that fits the mandatory **FULL JSON** format. This rigorous structure ensures consistent computation of metrics across all providers and baselines.

```
{
    "result_set": [
        { "originaltitle": "The Three Musketeers"},
        { "originaltitle": "The Count of Monte Cristo"}
    ],
    "time": 1.84,
    "tokens": 312
}
```

Figure 1: `FULL JSON` Output Format Example

Field Name	Description
result_set	Denotes the structured LLM output. This must be an array of JSON objects corresponding to the query's selected columns.
time	Represents the total time taken by the LLM to generate the response.
tokens	Indicates the number of tokens processed during the interaction.

Table 1: Definition of the `FULL JSON` Output Schema Fields

3 Specific Baseline Implementation

Leveraging the unified architecture described in Section 2.2, the two baselines differ almost exclusively in the type of input provided to the LLM during the "Prompt Construction" phase.

3.1 Baseline NL (Natural Language)

The objective of the **Natural Language (NL) Baseline** is to evaluate the LLM's intrinsic reasoning capabilities and its capacity to infer structured intent starting exclusively from **unstructured natural language input**.

- **Core Input**: Natural language questions are loaded from dataset-specific resource files (e.g., `queries_<dataset-name>.txt`).
- **Prompt Composition**: The final user input is constructed by combining two components: the **natural language question** and the essential `database schema` information.
- **Primary Goal**: Assess the model's ability to translate high-level, unstructured requests into a structured result based on its internal knowledge and the provided schema context.

3.2 Baseline SQL

The objective of the **SQL Baseline** is to strictly assess the LLM's capacity to interpret **complex SQL syntax**, perform logical reasoning over the supplied schema, and ensure the resultant output adheres to the required structured format.

- **Core Input**: Structured SQL queries are systematically loaded from dedicated files (e.g., `queries_<dataset-name>.sql`).
- **Prompt Composition**: The user input is populated directly with the **SQL statement** itself. The `database schema` is added as supplementary, explicit knowledge to guide and validate the model's output generation.
- **Primary Goal**: Quantify the performance when the structured intent is explicitly provided, testing adherence to syntactical and logical constraints.

3.3 Execution Control via Command-Line Interface (CLI)

The execution system is designed for flexible operation, allowing the user to precisely control which baselines run and which datasets and providers are targeted.

High-Level Execution Flow The overall sequence is strictly dictated by the CLI arguments:

1. **Initialization and Parsing**: Command-line arguments are parsed to determine the target datasets, execution mode (`-mode`), and preferred LLM provider (`-provider`). CLI input takes precedence over default configuration settings.

2. **Conditional Invocation**: Based on the selected `-mode` (`nl`, `sql`, or `both`), the system conditionally dispatches the execution logic to the corresponding baseline implementation.

3. **Task Dispatch and Persistence**: The provider-agnostic execution task is initiated, and the resulting `FULL_JSON` output files are persistently stored in the designated evaluation directory.

CLI Argument Reference The system accepts the following arguments for runtime control:

Argument	Type	Function / Description
`datasets`	Positional (List)	Accepts zero to multiple dataset identifiers to be processed (e.g., `WORLD`, `GEO`, `WINE`).
`-mode`	Optional (Flag)	Selects the execution type: `nl`, `sql`, or `both`.
`-provider`	Optional (Flag)	Overrides the default LLM provider defined in the configuration file (`gemini` or `watsonx`).

Table 2: Command-Line Interface Arguments

Note that if the datasets list is omitted, the system defaults to processing all available datasets as defined in the **config.yaml** fil, same for the provider.

4 Results and Quantitative Evaluation

The final quantitative outcomes for the SQL and NL Baselines are integrated below, detailing the core performance metrics: `F1-CELL`, `CARDINALITY`, and `TUPLE-CONSTRAINT`. These are complemented by operational metrics: `Average Score`, `Average Inference Time`(Latency), and total `Token Consumption`.

4.1 Evaluation Scope and Model Configuration

The results were obtained using the following LLM providers and models:

◇ **Gemini Provider**: `gemini-2.5-flash`

◇ **IBM Watsonx Provider**: `meta-llama/llama-3-70b-instruct`

All systems were tested exclusively on the datasets described in Table 2 of the previous project task, specifically **excluding Premier and Fortune** datasets, consistent with the methodology established by the original Galois authors.

4.1.1 Operational Consideration: Rate Limiting

Crucially, the use of the Gemini free-plan necessitated handling API minute limitations. Consequently, the observed running time for the Gemini baseline may be inflated compared to Watsonx. To ensure robustness and completeness, a simple **backoff retry mechanism** was implemented to pause and retry the request upon encountering an 'out of quota' error from the Gemini API.

4.2 Quantitative Results Comparison

The performance of each LLM across the Natural Language (NL) and Structured Query Language (SQL) contexts is summarized in the tables below:

Metric	NL	SQL
F1-CELL	0.424	0.380
CARDINALITY	0.705	0.506
TUPLE-CONSTR.	0.262	0.168
#TOKENS (M)	0.031	0.019
AVG-TIME (s)	5.706	6.476
AVG-SCORE	**0.464**	**0.351**

Table 3: GEMINI Results

Metric	NL	SQL
F1-CELL	0.263	0.492
CARDINALITY	0.494	0.492
TUPLE-CONSTR.	0.090	0.139
#TOKENS (M)	0.004	0.003
AVG-TIME (s)	1.715	1.529
AVG-SCORE	**0.282**	**0.330**

Table 4: WATSONX Results

4.3 Discussion of Key Findings

The analysis of results across different providers and input modalities highlights several intrinsic factors influencing performance and cost metrics:

- **Model Precision and Fluctuations**: Quality metrics are intrinsically linked to the precision and reasoning ability of the underlying LLM. The observed fluctuation in key metrics (e.g., TUPLE-CONSTRAINT) across providers and input types underscores the model's varied sensitivity to prompt structure and task complexity.
- **Impact of False Negatives**: A critical factor influencing metric scores is the LLM's propensity to generate values or parameter names that are semantically similar but syntactically divergent from the expected attribute (first of all in NL prompts) names found in the ground truth. This minor discrepancy leads to an increase in false negatives, suppressing the overall scores.
- **Prompt Quality as a Performance Driver**: The quality metrics obtained are strongly dependent on the quality and optimization of our Prompting. The current results suggest that our Prompt, while effective in guiding the response, is likely inferior in terms of few-shot examples or chain-of-thought compared to more advanced strategies, such as those adopted in the Galois framework.
- **Cost and Efficiency Variability**: Significant variability is noted in operational costs, specifically the number of tokens consumed (#TOKENS(M)) and the average execution time (AVG-TIME (s)). These differences are primarily attributed to distinct internal prompting strategies, context handling limitations, and inherent API latency variations across providers. Furthermore this particular metric shows that Gemini is much more verbose: Instead of giving just the dry answer (e.g., the SQL query or the extracted data), it likely generates complete sentences with more text than watsonx which is more effective and efficient giving results with minimal resource consumption.

5 Future Work and Strategic Development

For the next project phase, we will focus on significantly improving and extending the current logical and structural data retrieval processes. The primary strategic effort involves the integration and deployment of the **GALOIS framework** to enhance the accuracy and efficiency of structured data retrieval.

This integration is critical for addressing the inherent limitations of directly prompting LLMs with complex Natural Language (NL) or SQL queries, which often lead to sub-optimal precision and recall.

5.1 GALOIS Framework Integration: Core Optimization Areas

Our implementation effort will concentrate on two main areas essential to the GALOIS architecture:

5.1.1 Logical Plan Optimization (Filter Push-Down)

We will implement the specialized **Filter-LLMScan** operator, designed to push selection conditions directly down to the LLM. This step is methodologically complex: pushing down all conditions reduces token cost but often degrades result quality due to the LLM's inherent difficulty in processing overly complex simultaneous requests. The optimization will determine the ideal subset of conditions to push down.

5.1.2 Physical Operator Design and Evaluation

We will implement and quantitatively evaluate both alternative physical scan operators supported by GALOIS, which represent distinct trade-offs between cost and result quality:

- ◇ **Table-Scan**: A token-efficient approach where a single prompt requests the LLM to return the entire result set in the structured JSON format. This method risks lower quality but conserves tokens.
- ◇ **Key-Scan**: A two-step, quality-focused operator. It first retrieves the key values for relevant tuples and then iteratively queries the LLM to populate the remaining attributes for each retrieved key.

Strategic Decision Mechanism: To effectively choose between the Table-Scan and Key-Scan strategies, we will incorporate GALOIS's confidence threshold (τ) mechanism. By computing the LLM's confidence in retrieving key values, we can select **Key-Scan** when confidence is high (prioritizing quality) or revert to **Table-Scan** when confidence is low (for more reliable, albeit potentially less complete, data retrieval).

5.2 Retrieval-Augmented Generation (RAG) System Base

A foundational `Retrieval-Augmented Generation` system has already been implemented to serve as a solid base for future tasks requiring external knowledge integration.

The implemented RAG system follows a standard, robust procedure:

1. **Data Ingestion**: Read and collect documents from a specific text repository folder (e.g., `.md`, `.txt` files).
2. **Embedding Generation**: Process the raw files by chunking the text and converting these segments into numerical vectors (**embeddings**) using a Hugging Face model.
3. **Vector Storage**: Create an in-memory vector database (using FAISS) to enable fast similarity search.
4. **Retrieval and Augmentation**: Upon receiving a user prompt, the system queries the vector database to retrieve the text chunks most relevant to the question, which are then used to augment the LLM's final prompt.

6 Contributions

Alessio Demo (Badge number: 2142885)

- **First Deadline:** Developed function database connection and instance creation of DuckDB database manager for each dataset using the ingest files (.duckdb files). Developed function for retrieving the SQL queries from .sql files, processing the queries and executing them on DuckDB database instance and finally store it in a json file.
- **Second Deadline:** Implemented the Baseline NL function for interacting with the LLM with a backoff in case of gemini returns a free-plan limitation exception, save the results in FULL JSON format and next using it in the main script. Unit tests for the baseline_tools script using pytest. RAG task implementation using external libraries for creating the embeddings and evaluate the similarity between them in relation to a given prompt. Developed scripts for getting the db schema from DuckDB database for each dataset and the SQL query parsing, that will be used in the GALOIS implementation in the next tasks.

Writing the reports sections reporting the complete pipeline followed for developing the stuff above.

Francesco Pivotto (Badge number: 2158296)

- **First Deadline:** Implemented a modular project structure by relocating core logic to `src/` and consolidating configuration and database management logic within dedicated utility modules (`config/loaders.py`). Established the framework for multi-provider support by adding initial integration for the `Grok LLM`, including dedicated configuration points (`.env`, `config.yaml`, `loaders.py`).
- **Second Deadline:** Completed initial implementation of the SQL baseline, including the core logic for connecting to `DuckDB` and structuring query execution via the Watsonx provider. Fixed critical issues in environment variable loading, ensuring robust and standardized retrieval of secrets and settings via the central configuration object (using `loaders.py`). Modularized the `sql_baseline` module, incorporating the LCEL Pattern to prepare for dynamic LLM selection. Centralized general utility functions by moving helpers into `baseline_tools`. Introduced the dedicated `llm_wrapper` class to handle all LLM connection logic, API parameter management, and calls.

Giorgia Amato (Badge number: 2159999)

- **First Deadline:** Contributed to project initialization by setting up the repository structure, environment configuration and utility scripts. Added EXPLAIN and EXPLAIN ANALYZE support with JSON serialization, implemented JSON generation for query outputs, and improved logging and test utilities.
- **Second Deadline:** Developed the Gemini-based SQL baseline, including IK/MC query handling, schema and raw-data injection into prompts, JSON parsing through Pydantic, and error/timeout management. Refactored and extended unit tests for baseline tools and added new tests validating LLM interaction and Gemini SQL execution.